FAP60 SDK API Manual

1. FingerApi

`FingerApi` is the core interface for fingerprint collection and management, used for SDK initialization, device control, image capture, and device info retrieval.

1.1 Initialization

1.1.1 Get Instance

```
/**
 * Returns the singleton instance of FingerApi.
 *
 * @return FingerApi instance
 */
public static FingerApi getInstance()
```

1.1.2 Initialize SDK

```
/**
 * Initialize the SDK.
 *
 * @param context Context
 * @param logSwitch LOG_OPEN or LOG_CLOSE
 * @param onDeviceStateChangedListener DeviceStateChangedListener callback
 * @return true if initialization succeeds, false otherwise
 */
public boolean init(
    Context context,
    int logSwitch,
    DeviceStateChangedListener onDeviceStateChangedListener
);
```

> **Example:**

```
FingerApi.getInstance().init(
    getApplicationContext(),
    FapInitParam.LOG_OPEN,
    state -> {
        if (state == EnumDeviceState.HAVE_DEVICE || state ==
EnumDeviceState.DEVICE_ATTACHED){
            // openDevice();
        } else if (state == EnumDeviceState.DEVICE_DETACHED){
            // closeDevice();
        }
    }
);
```

1.2 Device Control

1.2.1 Open Device

```
/**
 * Turn on the fingerprint sensor.
 * NOTE: Do not call on main thread!
 *
 * @return file descriptor (>0) or negative error code
 */
@WorkerThread
int openDevice();
```

1.2.2 Close Device

```
/**
 * Turn off the fingerprint sensor.
 * NOTE: Do not call on main thread!
 *
 * @return 0 if successful, negative error code otherwise
 */
@WorkerThread
int closeDevice();
```

1.3 Capture

1.3.1 Get Image

```
/**
 * Capture an image from the fingerprint sensor.
 *
 * @param config Capture configuration {@link CaptureConfig}
 * @return MxResult<MxImage> containing captured image data
 */
@WorkerThread
MxResult<MxImage> getImage(CaptureConfig config);
```

1.3.2 Stop Capture

```
/**
 * Stop ongoing capture.
 *
 * @return 0 if successful, negative error code otherwise
 */
int stopCapture();
```

1.4 Device Info

```
MxResult<String> getDeviceVersion();       // Get device firmware version
MxResult<String> getDeviceSerialNumber();  // Get device serial number
MxResult<String> getSDKVersion();          // Get SDK version
```

2. CaptureConfig

`CaptureConfig` is the configuration object for fingerprint capture. **Only Builder pattern is exposed for external use.**

```
public class CaptureConfig {
    public static final int DEFAULT_AREA_SCORE = 20;
```

```
    /** Preview callback during capture */
    private FpPreviewCallBack previewCallBack;

    /** Number of missing fingers (auto-calculated) */
    private int missFingerNum;

    /** Capture type: LEFT_FOUR, RIGHT_FOUR, BOTH_THUMB, SINGLE, ROLL */
    private EnumPrintType printType;

    /** Actual fingers to capture (excluding missing fingers) */
    private ArrayList<EnumFingerPosition> captureFingers;

    /** Required NFIQ quality level (default = 3) */
    private int nfiqLevel = 3;

    /** Minimum acceptable fingerprint area score (default = 20) */
    private int areaScore = DEFAULT_AREA_SCORE;
}
```

Builder Configuration

Field	Type	Default	Description
previewCallBack	FpPreviewCallBack	null	Callback for real-time fingerprint preview
printType	EnumPrintType	LEFT_FOUR	Capture type: LEFT_FOUR, RIGHT_FOUR, BOTH_THUMB, SINGLE, ROLL
missingFingers	List	null	Fingers to exclude from capture
nfiqLevel	int	3	Required NFIQ quality threshold (1 = best quality, 5 = worst)
areaScore	int	20	Minimum acceptable fingerprint area score (higher = larger coverage)

Builder Methods

```
public class CaptureConfig.Builder {

    Builder setPreviewCallBack(FpPreviewCallBack previewCallBack)
    Builder setEnumPrintType(EnumPrintType printType)
    Builder setMissingFinger(List<EnumFingerPosition> fingers)
    Builder setNfiqLevel(int nfiqLevel)
    Builder setAreaScore(int areaScore)

    CaptureConfig build()
}
```

- `setPreviewCallBack` – set real-time preview callback
- `setEnumPrintType` – set capture type
- `setMissingFinger` – set fingers to exclude

- `setNfiqLevel` – set minimum acceptable NFIQ quality
- `setAreaScore` – set minimum acceptable fingerprint area score
- `build()` – generates a `CaptureConfig` instance, automatically computing actual fingers and missing fingers count

---

Builder Example

```
CaptureConfig config = new CaptureConfig.Builder()
        .setEnumPrintType(EnumPrintType.LEFT_FOUR)
        .setMissingFinger(Collections.singletonList(EnumFingerPosition.LEFT_INDEX))
        .setNfiqLevel(3)        // require NFIQ <= 3
        .setAreaScore(20)       // require fingerprint area >= 20
        .setPreviewCallBack((image, error) -> {
            // Handle preview frame
        })
        .build();

// Read actual captured fingers
ArrayList<EnumFingerPosition> fingers = config.getCaptureFingers();
int missing = config.getMissFingerNum();
```

---

3. EnumFingerPosition

`EnumFingerPosition` represents finger positions for capture or comparison.

Enum Name	Code	Description
UNKNOWN	0	Unknown finger
RIGHT_THUMB	1	Right thumb
RIGHT_INDEX	2	Right index
RIGHT_MIDDLE	3	Right middle
RIGHT_RING	4	Right ring
RIGHT_LITTLE	5	Right little
LEFT_THUMB	6	Left thumb
LEFT_INDEX	7	Left index
LEFT_MIDDLE	8	Left middle
LEFT_RING	9	Left ring
LEFT_LITTLE	10	Left little

Methods

```
int getCode();                        // Get finger code
static EnumFingerPosition fromName(String name); // Get enum from name
static EnumFingerPosition fromCode(int code);    // Get enum from code
```

4. MxImage

```
public class MxImage {
    public final int width;        // Image width
    public final int height;       // Image height
    public final int channels;     // Channels: 1=Grayscale, 3=RGB, 4=ARGB
    public final byte[] data;      // Image data (RAW or cropped fingerprint)
    public final EnumPrintType printType;  // Capture type
    public final int fingerNum;            // Number of fingers
    public final RawInfo[] singleFingerImages; // Single finger images extracted from the
raw image
    public final long[] time;              // Capture timestamp

    public static class RawInfo {
        public final byte[] imageData;         // Image bytes
        public final int width;
        public final int height;
        public final int nfiqLevel;            // NFIQ quality
        public final int area;                 // Finger Area
        public final EnumFingerPosition position; // Finger position
    }
}
```

5. MxResult

```
public class MxResult<T> {
    private final int code;   // Error code: 0=success, negative=failure
    private final String msg; // Error message
    private final T data;     // Data payload
    private final int sw1;    // Finger device error code
}
```

Notes:

1. Only `CaptureConfig.Builder` is exposed externally. Internal setters and constructors are hidden.
2. Builder automatically calculates actual fingers to capture based on capture type and missing fingers.
3. Supports real-time preview, NFIQ quality calculation.
4. EnumFingerPosition provides safe conversion from `name` or `code`.